

Program Information

I have uploaded my C++ project to the GitHub for this class (https://github.com/tgmaxson/HPC_TGM) under folder “hw3”. This is a modified form of HW1, which now supports OpenMP.

Cluster Information

I am running on Ubuntu 22.04.2 LTS via a cluster known as “Clustie” to our research group, which I have full access to. Tests are run on a test node which has nothing else running at the time of execution, except for SLURM and other background tasks which are default to Ubuntu Server installs. GCC is version 11.4.0 and ICC is version 2021.10.0 via OneAPI released on 06/09/2023. CUDA is provided via Nvidia HPC SDK version and nvcc is version cuda_12.0.r12.0/compiler.32267302_0 with CUDA version V12.0.140.

System Hardware

Motherboard: H11SSL-i CPU: Single Socket 32 Core AMD Epyc 7551P, 2.0 GHz, boost to 3.0 GHz. SMT Disabled. Memory: 256 GB of ECC DDR4-2400 via 8 x 32 GB sticks in 8 channel GPU: 2 x MSI Suprim Liquid X24G RTX 4090 with 24GB of memory.

Original HW3 Submission

Problems Identified:

- Arrays are allocated on the stack not the heap, ulimit required to be set
- Parallel region set twice, resulting in nested threading incorrectly

The following test is particularly short, but this seems to work for a quick judgement if it is working since it should be linear in time even at fairly small sizes. The goal here is just to identify when performance changes. Tests are performed on 16 cores.

Performance test:

```
EXE: ./bin/hw3 SIZE: 1000 GENS: 100 SEC: 0.253979
EXE: ./bin/hw3 SIZE: 1000 GENS: 500 SEC: 0.240597
EXE: ./bin/hw3 SIZE: 5000 GENS: 100 SEC: 5.295664
EXE: ./bin/hw3 SIZE: 5000 GENS: 500 SEC: 5.209338
EXE: ./bin/hw3 SIZE: 10000 GENS: 100 SEC: 21.956519
EXE: ./bin/hw3 SIZE: 10000 GENS: 500 SEC: 21.180676
```

Fixed array allocation

Arrays are now being allocated on the heap by allocating similar to sample.c provided as example. Arrays are also properly freed at end of program. In a fully proper program, we will also ensure that if the memory allocation fails, we do something special, but currently it is simply printed that it failed and the program will end up segfaulting.

Performance test:

```
EXE: ./bin/hw3 SIZE: 1000 GENS: 100 SEC: 0.257895
EXE: ./bin/hw3 SIZE: 1000 GENS: 500 SEC: 0.266482
EXE: ./bin/hw3 SIZE: 5000 GENS: 100 SEC: 5.226772
EXE: ./bin/hw3 SIZE: 5000 GENS: 500 SEC: 5.177519
EXE: ./bin/hw3 SIZE: 10000 GENS: 100 SEC: 22.201292
EXE: ./bin/hw3 SIZE: 10000 GENS: 500 SEC: 21.485131
```

Fixed parallel region

Parallel region is now fixed by inlining the update function and only calling the parallel directive once. The early break is eliminated from the code to avoid needing to update a flag in the parallel region, which was problematic. Additionally, the performance tests are changing now since it turns out `const clock_t start_time = clock();` was providing a threaded CPU time rather than realtime. Now the profiling is done via the following script.

```
#!/bin/bash
#SBATCH -J ProfileProgram
#SBATCH -N 1
#SBATCH -p main
#SBATCH --time=4:00:00
#SBATCH --ntasks-per-node=32

source ~/.bashrc
mamba activate work

gcc -o bin/hw3 hw3.c -lm -Wall -O3 -fopenmp

rm performance.txt

for threads in 1 2 4 8 16 20 24
do
  for size in 500 1000 2000
  do
    for gens in 10000
    do
      ./bin/hw3 $size $gens output-$size-$gens.txt $threads 0 | tee
performance.txt
    done
  done
done
```

The output is as follows.

```
EXE: ./bin/hw3, THREAD: 1, SIZE: 500, GENS: 10000, SEC: 2.312000
EXE: ./bin/hw3, THREAD: 1, SIZE: 1000, GENS: 10000, SEC: 9.248000
EXE: ./bin/hw3, THREAD: 1, SIZE: 2000, GENS: 10000, SEC: 37.449000
EXE: ./bin/hw3, THREAD: 2, SIZE: 500, GENS: 10000, SEC: 1.227000
EXE: ./bin/hw3, THREAD: 2, SIZE: 1000, GENS: 10000, SEC: 4.624000
EXE: ./bin/hw3, THREAD: 2, SIZE: 2000, GENS: 10000, SEC: 19.129000
EXE: ./bin/hw3, THREAD: 4, SIZE: 500, GENS: 10000, SEC: 0.650000
EXE: ./bin/hw3, THREAD: 4, SIZE: 1000, GENS: 10000, SEC: 2.445000
EXE: ./bin/hw3, THREAD: 4, SIZE: 2000, GENS: 10000, SEC: 9.739000
EXE: ./bin/hw3, THREAD: 8, SIZE: 500, GENS: 10000, SEC: 0.376000
EXE: ./bin/hw3, THREAD: 8, SIZE: 1000, GENS: 10000, SEC: 1.356000
EXE: ./bin/hw3, THREAD: 8, SIZE: 2000, GENS: 10000, SEC: 5.172000
EXE: ./bin/hw3, THREAD: 16, SIZE: 500, GENS: 10000, SEC: 0.239000
EXE: ./bin/hw3, THREAD: 16, SIZE: 1000, GENS: 10000, SEC: 0.811000
EXE: ./bin/hw3, THREAD: 16, SIZE: 2000, GENS: 10000, SEC: 3.045000
EXE: ./bin/hw3, THREAD: 20, SIZE: 500, GENS: 10000, SEC: 0.270000
EXE: ./bin/hw3, THREAD: 20, SIZE: 1000, GENS: 10000, SEC: 0.695000
EXE: ./bin/hw3, THREAD: 20, SIZE: 2000, GENS: 10000, SEC: 2.584000
EXE: ./bin/hw3, THREAD: 24, SIZE: 500, GENS: 10000, SEC: 0.200000
EXE: ./bin/hw3, THREAD: 24, SIZE: 1000, GENS: 10000, SEC: 0.588000
EXE: ./bin/hw3, THREAD: 24, SIZE: 2000, GENS: 10000, SEC: 2.212000
```

Performance now appears to scale based on threads and problem size properly!

